GeoClaw-claude

Python 城市地理信息分析工具集

技 术 文 档 · Technical Reference

版本 v2.2.1 · 2025 年 3 月

UrbanComp Lab — https://urbancomp.net

GitHub — https://github.com/whuyao/GeoClaw_Claude

MIT License | Copyright © 2025 UrbanComp Lab

1. 项目概述

1.1 背景与目标

GeoClaw-claude 是由 UrbanComp Lab（https://urbancomp.net）开发的轻量级 Python 城市地理信息分析工具集。项目参考 QGIS Processing Framework 的模块化架构，以 Python 原生生态为基础，面向城市地理空间数据分析场景，提供从数据获取、空间分析到可视化制图的完整工作流。

工具集名称来源于其核心定位：GeoSpatial Analysis + Claude AI 驱动的 Skill 系统与自然语言操作接口。自 v1.3.0 起引入自然语言操作系统，自 v2.1.0 起集成 trackintel 框架支持人类移动性分析。

1.2 主要特性

- 自然语言 GIS 操作（v1.3.0+）：30+ 种 GIS 操作支持中文自然语言驱动，AI 模式（Claude API）+ 规则模式（本地离线）双驱动，低置信度时主动确认
- 人类移动性分析（v2.1.0+）：基于 trackintel 框架，GPS 轨迹 → 停留点 → 出行段 → 出行 → 重要地点完整层级生成，支持自然语言调用
- 跨会话 Memory 系统（v1.1.0+）：短期记忆（TTL 缓存 + 操作日志）+ 长期记忆（JSON 持久化 + 多维检索），跨任务知识积累
- 自动更新与自检（v1.2.0+）：版本检测（读取 GitHub 最新 __version__）、健康检测（17 项）、一键 git pull + pip install 升级
- 完整 GIS 工具链：矢量/栅格/路网/制图，OSM、WFS、天地图多数据源
- 中国数据友好：WGS84/GCJ-02/BD-09 坐标互转，天地图 POI API
- 科研向设计：详细文档字符串、TODO 标注，方便研究人员二次开发

1.3 适用人群

用户类型	适用场景
科研人员 / 博士生	城市空间分析、POI 分布研究、路网可达性、GPS 移动性研究
实验室本科生	学习 GIS 编程、复现论文实验、完成课程设计

外部合作者	快速上手城市数据分析，使用内置 Skill 与 Demo 案例
工程开发者	二次开发、自定义 Skill、集成到已有数据处理流程

1.4 版本历史

版本	日期	主要更新
v2.2.1	2025-03	README 全面重组；移动性分析 NL 说明增强；NL 关键词映射修复（出行方式构成图/预测出行方式）
v2.2.0	2025-03	武汉 GPS 轨迹测试数据集（37,549 点）；完整 Demo 脚本；trackintel 算法来源声明
v2.1.0	2025-03	analysis/mobility/ 模块（trackintel 集成）；NL 新增 10 类移动性操作；测试 20/20
v2.0.0	2025-03	重大版本升级，自然语言 GIS 平台，全面文档同步，API 完全向下兼容 v1.x
v1.3.0	2025-03	自然语言操作系统（nl/ 模块）；CLI ask/chat 命令；NL 测试 20/20
v1.2.0	2025-03	自我检测与自动更新（updater.py）；CLI check/update/self-check；测试 20/20
v1.1.0	2025-03	Memory 记忆系统（memory/ 模块）；CLI memory 命令组（8 子命令）；测试 37/37
v1.0.0	2025-03	首个正式版本：CLI、Skill 框架、路网/栅格分析、坐标转换、武汉示例数据
v0.x	2025-02	原型阶段：GeoLayer 核心类、OSM 下载、空间分析骨架、CLI 框架

2. 工程结构

2.1 目录树

```
GeoClaw_Claude/
├── geoclaw_claude/
│   ├── __init__.py          # 版本信息（__version__ = '2.2.1'）
│   ├── cli.py               # Click CLI 入口（所有命令）
│   ├── config.py            # 配置系统（JSON + 环境变量）
│   ├── updater.py           # 版本检测与自动更新  ← v1.2.0
│   ├── skill_manager.py     # Skill 加载与运行
│   ├── core/
│   │   ├── layer.py         # GeoLayer 核心类
│   │   └── project.py       # GeoClawProject 项目管理
│   ├── analysis/
│   │   ├── spatial_ops.py   # buffer/clip/nn/kde
│   │   ├── network.py       # 路网分析
│   │   ├── raster_ops.py    # 栅格分析
│   │   └── mobility/        # 移动性分析  ← v2.1.0
│   │       ├── core.py      #   层级生成（trackintel 封装）
│   │       ├── metrics.py   #   指标计算
│   │       └── visualization.py  # 可视化（英文标签）
│   ├── nl/                  # 自然语言操作系统  ← v1.3.0
│   │   ├── processor.py     #   NLProcessor（AI+规则双模式）
│   │   ├── executor.py      #   NLExecutor（意图→GIS 执行）
│   │   └── agent.py         #   GeoAgent（多轮对话）
```

```
│   ├── memory/              # 记忆系统  ← v1.1.0
│   │   ├── short_term.py
│   │   ├── long_term.py
│   │   └── manager.py
│   ├── cartography/
│   ├── io/
│   └── utils/
├── data/
│   ├── wuhan/               # 武汉 GIS 示例数据（7 个 GeoJSON）
│   └── mobility/            # GPS 轨迹测试数据集  ← v2.2.0
│       ├── wuhan_gps_tracks.csv  # 37,549 个轨迹点
│       └── README.md
├── examples/
│   └── wuhan_mobility_demo.py   # 完整移动性分析 Demo
├── tests/
│   ├── test_memory.py    # 37 项
│   ├── test_updater.py   # 20 项
│   ├── test_nl.py        # 20 项
│   └── test_mobility.py  # 20 项
└── install.sh
```

2.2 测试覆盖

测试文件	通过/总数	覆盖范围
test_memory.py	37/37 ✓	ShortTermMemory / LongTermMemory / MemoryManager
test_updater.py	20/20 ✓	check / update / self_check / changelog_diff
test_nl.py	20/20 ✓	NLProcessor / NLExecutor / GeoAgent（AI+规则双模式）
test_mobility.py	20/20 ✓	读入/层级生成/指标/可视化/NL 调用/trackintel 兼容性

3. 自然语言操作系统（nl/ 模块）

nl/ 模块是 GeoClaw-claude v1.3.0 的核心新增特性。通过 NLProcessor（意图解析）→ NLExecutor（GIS 执行）→ GeoAgent（多轮对话）三层架构，实现用自然语言直接驱动 GIS 分析，无需记忆 API 函数名。

3.1 架构设计

三层架构如下：

- NLProcessor（processor.py）：自然语言 → ParsedIntent，支持 AI 模式和规则模式
- NLExecutor（executor.py）：ParsedIntent → GIS 函数调用，图层上下文管理，结果写入 Memory
- GeoAgent（agent.py）：封装多轮对话，跨轮保持图层上下文，会话结束自动写入长期记忆

3.2 NLProcessor — 意图解析

3.2.1 ParsedIntent 数据结构

```
@dataclass
```

```
class ParsedIntent:
    action:      str          # 操作类型，如 'buffer' / 'mobility_hierarchy'
    params:      dict         # 参数字典，如 {'radius': 1000, 'unit': 'meters'}
    targets:     list[str]    # 操作目标图层名列表
    confidence:  float        # 0.0~1.0，置信度
    explanation: str          # 人类可读解释，用于确认提示
    steps:       list[dict]   # 多步流水线时的分解步骤列表
```

3.2.2 AI 模式

配置 Anthropic API Key 后自动启用。调用 Claude API 理解语义，支持复杂意图（歧义消解、多步流水线、参数推断）。返回结构化 JSON，解析为 ParsedIntent。

3.2.3 规则模式

无 API Key 或指定 --rule 时使用。基于关键词词典 + 正则表达式本地解析，完全离线。v2.2.1 支持的规则操作类型如下：

action	触发关键词（部分）	说明
load	加载、读入、打开、load	加载本地或远程数据文件
buffer	缓冲区、buffer、半径、范围	空间缓冲区分析
clip	裁剪、clip、切割	空间裁剪
nearest_neighbor	最近邻、nearest、距离	最近邻距离计算
kde	核密度、kde、density、热力图	KDE 核密度估计
zonal_stats	分区统计、zonal、按...统计	按区域汇总统计
isochrone	等时圈、isochrone、可达性	路网等时圈
shortest_path	最短路径、路径规划、shortest	路网最短路径
coord_transform	坐标转换、wgs84、gcj02、bd09	坐标系互转
mobility_load	轨迹数据、gps、positionfix	读入 GPS 移动性数据
mobility_staypoints	停留点、staypoint、驻留	生成停留点
mobility_hierarchy	一键完成移动性分析、全层级	一键生成完整层级
mobility_transport	预测出行方式、出行方式预测	预测交通方式
mobility_locations	重要地点、识别家和工作地	地点聚类与语义识别
mobility_summary	移动性指标摘要、回转半径	计算移动性综合指标
mobility_plot	轨迹地图、mobility_plot	分层移动性地图
mobility_heatmap	时间热力图、活动热力图	活动时间热力矩阵
mobility_modal	出行方式构成图、modal	出行方式构成图
render	可视化、地图、显示	通用可视化

3.3 NLExecutor — GIS 执行

NLExecutor 接收 ParsedIntent，映射到对应 geoclaw_claude 函数并执行，返回 ExecutionResult。核心设计：

- 图层上下文（layer_context）：dict 存储命名图层，跨轮自动传递。加载的数据自动命名，如 hospitals、buffer_hospitals

- 操作链：通过 steps 列表实现多步流水线，前一步结果自动传入下一步
- Memory 集成：每次操作自动调用 mem.log_op()，成功结果自动 remember()
- 置信度阈值：confidence < 0.65 时自动触发确认提示，不直接执行

ExecutionResult 数据结构

```
@dataclass
class ExecutionResult:
    success:     bool
    result:      Any        # 执行结果（图层、数值、字典等）
    message:     str        # 人类可读结果描述
    duration_ms: float      # 执行耗时（毫秒）
    layer_name:  str|None   # 输出图层名（若有）
```

3.4 GeoAgent — 多轮对话

```
from geoclaw_claude.nl import GeoAgent

agent = GeoAgent()            # 自动选 AI / 规则模式
agent.chat("加载 data/wuhan/hospitals.geojson")
# ✓ 已加载 200 个要素  耗时: 0.12s

agent.chat("对医院做 1 公里缓冲区")
# ✓ 缓冲区完成，200 个要素，半径 1000.0m  耗时: 0.43s

agent.chat("用交互地图显示结果")
# ✓ 地图生成完成

agent.print_history()                      # 打印对话历史
agent.end(title='武汉医院缓冲区分析')     # 写入长期记忆
```

低置信度时，代理自动请求确认：

```
agent.chat("差不多做个热力图那种东西")
# 我理解你想要: KDE 核密度分析（置信度 52%）
# 是否执行? (是/否)
agent.chat("是")
# ✓ 核密度分析完成，100×100 网格
```

3.5 CLI 命令详解

命令	说明
geoclaw-claude ask "..."	单条自然语言指令，解析并执行
geoclaw-claude ask --dry-run "..."	只解析意图，输出 JSON，不执行
geoclaw-claude ask --rule "..."	强制规则模式（离线，无需 API Key）
geoclaw-claude ask --ai "..."	强制 AI 模式（调用 Claude API）
geoclaw-claude chat	进入交互式多轮对话模式
geoclaw-claude chat --ai	交互式对话，强制 AI 模式

4. Memory 记忆系统（memory/ 模块）

Memory 系统为 GeoClaw-claude 提供跨会话的知识积累能力，分为短期（会话内缓存）和长期（持久化存储）两层，通过 MemoryManager 全局单例统一管理。

4.1 ShortTermMemory

4.1.1 核心功能

- TTL 缓存：remember(key, value, ttl) 存储中间结果，超时自动失效
- 操作日志：log_op(func_name, args_summary) 自动记录每步操作（含时间戳、耗时、成功/失败）
- 上下文管理：set_context(key, value) / get_context(key) 在单次会话内共享状态
- 线程安全：RLock 保护所有读写操作，支持并发场景

4.1.2 API 参考

```
mem = get_memory()
mem.start_session('session_name')

mem.log_op('buffer', 'hospitals, 1km')  # 记录操作
mem.remember('buf_hospitals', layer)     # 缓存结果（默认 TTL=3600s）
layer = mem.recall_short('buf_hospitals')
mem.set_context('city', 'wuhan')
city = mem.get_context('city')
```

4.2 LongTermMemory

4.2.1 存储结构

持久化到 ~/.geoclaw_claude/memory/，每条记忆包含：

- id（UUID）、title、content（dict）、category、tags（list）、importance（0~1）
- created_at、updated_at、access_count（访问次数），支持自动重要度衰减

4.2.2 API 参考

```
mem.learn(
    title='武汉医院分布规律',
    content={'finding': '医院主要集中在三环内，外围覆盖不足'},
    tags=['wuhan', 'hospital', 'spatial'],
    category='knowledge',
    importance=0.8,
)

mem.recall('武汉 医院')             # 关键词全文检索
mem.recall_recent(n=5)              # 最近 5 条记忆
mem.recall_important(n=5, threshold=0.7)  # 重要度 ≥ 0.7
mem.recall_by_tag('hospital')       # 按标签检索
```

4.3 会话复盘（end_session）

mem.end_session() 在会话结束时自动将本次分析过程 flush 到长期记忆：

- 操作序列：按时间顺序记录所有 log_op()

- 统计摘要：总操作数、成功/失败数、总耗时
- 中间结果清单：所有 remember() 的键名列表

4.4 CLI 命令

命令	说明
geoclaw-claude memory status	显示系统状态（短期/长期条目数、存储路径）
geoclaw-claude memory list	列出所有长期记忆条目
geoclaw-claude memory list -c knowledge	按类别筛选
geoclaw-claude memory search "武汉 医院"	关键词全文检索
geoclaw-claude memory learn TITLE CONTENT	手动存入知识
geoclaw-claude memory forget ENTRY_ID	删除指定记忆条目
geoclaw-claude memory compact	压缩旧记忆（合并低重要度条目）
geoclaw-claude memory export -o out.json	导出全部长期记忆为 JSON

5. 人类移动性分析（analysis/mobility/）

analysis/mobility/ 模块自 v2.1.0 引入，封装 **trackintel**（https://github.com/mie-lab/trackintel，ETH Zurich）框架，提供从原始 GPS 轨迹到移动性指标的完整分析流水线，并通过 NL 模块支持自然语言调用。

5.1 数据层级模型

trackintel 将人类移动性数据抽象为 5 个层级，GeoClaw-claude 提供对应封装函数：

数据层级	生成函数	算法说明
positionfixes	read_positionfixes()	读入原始 GPS 轨迹点，规范化 CRS 与时区，支持 CSV / GeoDataFrame / DataFrame
staypoints	generate_staypoints()	滑动窗口法：dist_threshold（空间半径）+ time_threshold（最短停留时间）双阈值判定停留
triplegs	generate_triplegs()	提取相邻停留点间的连续出行段；可选 predict_transport_mode() 标注交通方式
trips	generate_trips()	聚合活动型停留点间的完整出行；需要 is_activity 列（v2.1.0 自动补充）
locations	generate_locations()	DBSCAN 空间聚类，location_epsilon 控制聚类半径；identify_home_work() 识别家/工作地

5.2 核心函数 API

5.2.1 read_positionfixes()

```
pfs = read_positionfixes(
    source,                # 路径字符串 / DataFrame / GeoDataFrame
    user_id_col='user_id',
```

```
    tracked_at_col='tracked_at',
    lon_col='longitude',
    lat_col='latitude',
    crs='epsg:4326',        # 默认 WGS84
)
```

注意：tracked_at 时间戳需包含时区信息（UTC），内部使用 format='mixed' 兼容 ISO8601 含微秒格式。

5.2.2 generate_full_hierarchy()

```
hierarchy = generate_full_hierarchy(
    pfs,
    dist_threshold=80,      # 停留判定空间半径（米，默认 80）
    time_threshold=5,       # 停留判定最短时间（分钟，默认 5）
    gap_threshold=30,       # 轨迹缺口容忍（分钟，默认 30）
    location_epsilon=120,   # 地点聚类半径（米，默认 120）
    predict_mode=True,      # 是否预测交通方式
)
```

返回 dict，包含：positionfixes / staypoints / triplegs / trips / locations。

5.2.3 predict_transport_mode()

基于速度特征的简单分类器（simple-coarse 方法）：

模式	速度区间（约）	典型交通方式
slow_mobility	< 15 km/h	步行、骑行
motorized_mobility	15~80 km/h	公共汽车、私家车、地铁（市区）
fast_mobility	> 80 km/h	高铁、飞机

5.2.4 identify_home_work()

```
locs_labeled = identify_home_work(
    staypoints,
    locations,
    method='osna',    # 'osna'（作息时间规律）或 'freq'（频率法）
)
```

OSNA 方法：家 = 夜间（22:00~06:00）停留频率最高的地点；工作地 = 工作日白天（09:00~17:00）停留频率最高的地点。

5.3 移动性指标

函数	说明
radius_of_gyration(sp)	回转半径（km）：用户所有停留点到质心的加权 RMS 距离，衡量活动范围大小。trackintel 输出列名 radius_gyration
jump_lengths(sp)	跳跃距离（km）：相邻停留点间的直线距离分布，描述单次出行距离特征
modal_split(tpls)	出行方式构成：按 mode 列统计次数、时长（分钟）、距离三种口径
tracking_quality(pfs/sp)	时间覆盖率（0~1）：有效轨迹时间 / 总观测时间，评估数据完整性
identify_home_work(sp, locs)	语义识别：为 locations 的 purpose 列标注 'home' / 'work' / None

mobility_summary(hierarchy)	综合摘要：自动计算以上所有指标并汇总为 dict

5.4 可视化函数

函数	输出说明
plot_mobility_layers(hierarchy)	分层移动性地图（暗色主题）：停留点（橙色圆点）+ 出行段（按交通方式上色）+ 重要地点（红色大圆）叠加
plot_activity_heatmap(sp)	活动时间热力矩阵（Weekday × Hour），YlOrRd 色阶，支持单用户 / 全用户模式
plot_modal_split(tpls)	出行方式构成图（饼图 + 水平条形图双联图）
plot_mobility_metrics(summary)	指标仪表盘（4 个子图）：数据集概览 / 回转半径柱图 / 出行方式饼图 / 轨迹覆盖率仪表盘

所有可视化函数均支持 save_path 参数保存 PNG（dpi=150），标签均为英文（避免中文字体兼容性问题）。

5.5 Demo 数据集详解

data/mobility/wuhan_gps_tracks.csv

字段	类型	说明
id	int	轨迹点唯一 ID（索引）
user_id	int	用户 ID（0~4）
tracked_at	datetime(UTC)	记录时间戳（ISO8601 含微秒，UTC 时区）
longitude	float64	经度，WGS84（范围约 114.17~114.42）
latitude	float64	纬度，WGS84（范围约 30.51~30.62）
elevation	float64	海拔（米，模拟值，仅参考）
accuracy	float64	定位精度（米，模拟值，3~20m）

5 位用户背景

ID	职业	居住地	工作地	通勤方式
u0	金融从业者	汉口后湖	武昌中南路	地铁
u1	互联网工程师	武昌洪山广场	光谷软件园	公交（弹性上班）
u2	商贸从业者	汉阳知音湖	汉口江汉路	步行
u3	高校研究员	武昌光谷	武汉大学	骑行
u4	医疗从业者	汉口新华路	协和医院	地铁

处理结果参考值（默认参数）

层级	数量	备注
positionfixes	37,549	5 用户 × 10 天，每用户约 7,100~8,100 个点

staypoints	~1,781	dist=80m, time=5min 判定停留
triplegs	~1,240	含 mode 列（89.9% slow_mobility，10.1% motorized）
trips	~1,240	
locations	~20	location_epsilon=120m DBSCAN 聚类

5.6 算法来源声明

analysis/mobility/ 中的核心轨迹处理算法来自 **trackintel**（ETH Zurich · Mobility Information Engineering Lab）。GeoClaw-claude 提供 API 封装、GeoLayer 生态集成、自然语言接口和可视化；原始算法实现归属 trackintel 项目及其作者。

- GitHub：https://github.com/mie-lab/trackintel
- 论文：Martin, H., Hong, Y., Wiedemann, N., Bucher, D., & Raubal, M. (2023). Trackintel: An open-source Python library for human mobility analysis. Computers, Environment and Urban Systems, 101, 101938.
- DOI：https://doi.org/10.1016/j.compenvurbsys.2023.101938

6. 自我检测与自动更新（updater 模块）

updater.py 自 v1.2.0 引入，提供版本检测、一键升级、全面健康检测三大功能。

6.1 版本检测

check() 函数从 GitHub 远程读取最新 __version__，与本地比对，返回 UpdateCheckResult：

```
from geoclaw_claude.updater import check, print_check_result
result = check()
print_check_result(result)
# 或 JSON: check(as_json=True)
```

6.2 自动更新

update() 执行 git pull + pip install -e .，并可选运行测试验证：

```
from geoclaw_claude.updater import update
update(test=True)   # 更新后运行 pytest tests/
```

6.3 健康检测报告

self_check() 运行 17 项检测，覆盖：版本状态、18 个模块导入、12 个依赖包、数据目录、配置文件。

```
from geoclaw_claude.updater import self_check, print_self_check
report = self_check(quick=False)   # quick=True 跳过网络检测
print_self_check(report)
# ╔══ GeoClaw-claude Self-Check ═══════════════╗
# ╟ Version      local: v2.2.1  remote: v2.2.1 ✓
# ╟ Modules      18/18 OK
# ╟ Dependencies 12/12 OK
# ╚═══════════════════════════════════════════╝
```

7. 其他功能模块

7.1 空间分析（analysis/spatial_ops.py）

函数	说明
buffer(layer, radius)	缓冲区分析。自动选择 UTM 投影（metric 单位精准计算），支持 unit='meters'/'km'
clip(layer, mask)	空间裁剪。返回 mask 范围内的要素子集
nearest_neighbor(src, dst)	最近邻距离计算。修复零距离 BUG（UTM 投影精准），返回包含 nn_distance 列的 GeoDataFrame
kde(layer, bandwidth, grid_size)	KDE 核密度估计（scipy.stats.gaussian_kde）。返回栅格化密度矩阵 + 空间范围
overlay(layer1, layer2, how)	空间叠加。how='intersection'/'union'/'difference'/'symmetric_difference'
zonal_stats(zones, points)	分区统计。按 zones 多边形统计 points 落点数，支持自定义聚合函数

7.2 路网分析（analysis/network.py）

函数	说明
build_network(bbox, type)	基于 OSMnx 构建路网图（NetworkX DiGraph），type='walk'/'bike'/'drive'
shortest_path(G, orig, dest)	Dijkstra 最短路径（权重='travel_time'），返回路径 GeoDataFrame + 距离/时间
isochrone(G, center, minutes)	等时圈分析，返回多个时间阈值对应的可达多边形 GeoDataFrame
service_area(G, sources, minutes)	批量服务区：多起点等时圈的并集，用于设施覆盖分析
network_stats(G)	路网统计：节点数、边数、平均度、聚类系数、连通分量数等

7.3 栅格分析（analysis/raster_ops.py）

函数	说明
load_raster(path)	加载 GeoTIFF，返回 (data, transform, crs) 三元组
slope(dem)	坡度（degrees），基于 DEM 数组 + 空间分辨率计算
aspect(dem)	坡向（degrees，0~360，北为 0）
hillshade(dem)	山体阴影，可调方位角（azimuth）与高度角（altitude）
reclassify(raster, rules)	栅格重分类，rules 为 [(min, max, new_val), ...] 列表
raster_calculator(expr, bands)	栅格计算器，支持表达式如 '(b1 - b2) / (b1 + b2)'（NDVI 等）
zonal_stats_raster(zones, raster)	分区栅格统计：min/max/mean/sum/std/count
resample(raster, scale)	重采样，scale > 1 为上采样（细化），< 1 为下采样（粗化）

7.4 坐标转换（utils/coord_transform.py）

提供 WGS84 / GCJ-02 / BD-09 三种坐标系的完整互转，支持单点和批量图层转换：

```
from geoclaw_claude.utils.coord_transform import wgs84_to_gcj02, transform_layer

lon, lat = wgs84_to_gcj02(114.3, 30.5)         # 单点
gcj_layer = transform_layer(layer, 'wgs84', 'gcj02')  # 批量图层
```

支持的转换路径：wgs84 ↔ gcj02 ↔ bd09，以及 wgs84 ↔ bd09 直接互转。

7.5 Skill 脚本系统

用户可编写符合规范的 Python Skill 脚本，通过 SkillContext 接口调用 Claude AI API 进行数据解读。

```
# my_skill.py
SKILL_META = {'name': 'my_analysis', 'version': '1.0.0', 'author': 'you'}

def run(ctx):
    layer     = ctx.get_layer('input')
    radius_km = float(ctx.param('radius_km', 5.0))
    ai_result = ctx.ask_ai('请分析这些数据的空间分布特征')
    return ctx.result(output=result_layer)
```

```
geoclaw-claude skill install my_skill.py
geoclaw-claude skill run my_analysis --data data.geojson --ai
```

8. 安装与配置

8.1 系统要求

- Python 3.9+（推荐 3.10/3.11/3.12）
- Git（用于自动更新功能）
- trackintel ≥ 1.4.2（移动性分析模块）
- Anthropic API Key（可选，NL AI 模式 + Skill AI 功能）

8.2 安装步骤

```
git clone https://github.com/whuyao/GeoClaw_Claude.git
cd GeoClaw_Claude
bash install.sh              # 自动创建虚拟环境并安装依赖
geoclaw-claude onboard       # 交互式初始化向导
```

8.3 手动安装依赖

```
pip install geopandas osmnx rasterio trackintel scipy matplotlib folium click
pip install anthropic          # 可选: Claude AI 接口
```

8.4 配置文件

配置存储于 ~/.geoclaw_claude/config.json，通过 CLI 管理：

```
geoclaw-claude config set anthropic_api_key YOUR_KEY
```

```
geoclaw-claude config set data_dir /path/to/data
geoclaw-claude config show
```

9. 关于

GeoClaw-claude 由 UrbanComp Lab（https://urbancomp.net）开发，服务于城市计算与地理信息科学研究。

GitHub 仓库：https://github.com/whuyao/GeoClaw_Claude

问题反馈：GitHub Issues

MIT License · Copyright © 2025 UrbanComp Lab